Git & GitHub for DevOps Complete Curriculum

Approach 2: Project-Driven ("Build-Along")

Beginner-friendly | 5 Sessions | 1 Continuous Project | Learn by Building

---

Instructor Guide

Philosophy

Students learn by **building one real project from start to finish**. Every session adds a feature to the project. By Session 5, they have a fully version-controlled website with CI/CD. The project IS the curriculum.

Magic Rule: No abstract examples — every command serves the project.

Pacing:
- 10 min: Project status check + plan for today
- 15 min: New concept (introduced because the project needs it)
- 25 min: Hands-on building (instructor demos, students follow)
- 10 min: Extend the feature (student challenge)

Student prerequisites: None.

The Project: `my-devops-portfolio` — a simple portfolio website. By Session 5 it has:
- Multiple pages (Home, About, Projects, Contact)
- Branch-based feature development
- GitHub collaboration workflow
- Automated CI testing
- Branch protection

Before You Start

Complete Environment Setup before Session 1.

Environment Setup Guide

Identical for all three approaches.

Step 1: Install Git

Environment: Windows

1. `https://git-scm.com/download/win`
2. Install with all defaults
3. Open **Git Bash**
4. Verify:

```
$ git --version
```

Expected Output:

```
git version 2.43.0.windows.1
```

Environment: Mac

```
$ git --version
```

If not installed, macOS will prompt to install Developer Tools.

Environment: Linux (Ubuntu/Debian)

```
$ sudo apt update && sudo apt install git -y
$ git --version
```

Environment: Linux (Fedora)

```
$ sudo dnf install git -y
$ git --version
```

Step 2: Configure Git

Environment: Any terminal

```
$ git config --global user.name "Your Full Name"
$ git config --global user.email "youremail@example.com"
```

Step 3: Create GitHub Account

Environment: Browser

1. `https://github.com`
2. Sign up with the same email
3. Verify email

Step 4: SSH Keys

Environment: Terminal

```
$ ssh-keygen -t ed25519 -C "youremail@example.com"
```

Press Enter three times.

Copy key:

Mac: `pbcopy < ~/.ssh/id_ed25519.pub`
Windows: `cat ~/.ssh/id_ed25519.pub | clip`
Linux: `cat ~/.ssh/id_ed25519.pub` (copy manually)

Add to GitHub: Settings → SSH and GPG keys → New SSH key

Test:

```
$ ssh -T git@github.com
```

Type `yes`.

Expected Output:

```
Hi yourusername! You've successfully authenticated...
```

Step 5: Install VS Code

Download from `https://code.visualstudio.com`.

The Project: `my-devops-portfolio`

We will build a personal portfolio website across 5 sessions. Here's the roadmap:

Session	Feature Added	Git Skill
1	Home page	Init, add, commit, status, log
2	About + Projects pages	Branching, merging
3	Push to GitHub + Contact page	Remotes, push, pull, PRs
4	Fix a bug + Style updates	Revert, resolve conflicts
5	Auto-deploy check	CI/CD with GitHub Actions

Session 1: Foundation — Home Page and First Commits

The Goal Today

Create the project folder, initialize Git, and build the Home page. Save your progress using commits.

Theory: What We Need to Know (15 minutes)

The Analogy — Building a House

> ***Analogy:*** *Building a project without Git is like building a house without taking photos. If the wall collapses, you have no idea what it looked like before.*
>
> *Git is your construction photographer. Every time you finish a room, you take a photo (commit). If the roof leaks, you can look at the photos and figure out when it started.*

The Three Stages

Think of Git like a restaurant kitchen:
1. **Working Directory** = The prep table (you're cutting vegetables)
2. **Staging Area** = The plate ready to be served (chef has arranged it)
3. **Commit** = The photo for the menu (permanent record)

You don't take menu photos of half-chopped vegetables. You stage first, then commit.

Practical Build: Home Page (35 minutes)

Environment: Terminal

Create the project:

```
$ cd ~
$ mkdir my-devops-portfolio
$ cd my-devops-portfolio
```

Initialize Git:

```
$ git init
```

Expected Output:

```
Initialized empty Git repository in .../.git/
```

Create the homepage:

```
$ cat > home.txt << 'EOF'
================================
  MY DEVOPS PORTFOLIO
================================

Hi, I'm [Your Name]!

I'm learning DevOps and this is my first Git project.

Visit the About page to learn more.
EOF
```

Check status:

```
$ git status
```

Expected Output:

```
Untracked files:
    home.txt
```

Stage it:

```
$ git add home.txt
```

Check status again:

```
$ git status
```

Expected Output:

```
Changes to be committed:
    new file:   home.txt
```

Commit:

```
$ git commit -m "Add homepage with welcome message"
```

Expected Output:

```
[main (root-commit) abc1234] Add homepage with welcome message
 1 file changed, 10 insertions(+)
```

Check history:

```
$ git log --oneline
```

Expected Output:

```
abc1234 Add homepage with welcome message
```

Now add a navigation section:

Environment: Text Editor

Open `home.txt` and add to the bottom:

```
---
Navigation: Home | About | Projects | Contact
---
```

Or via terminal:

```
$ cat >> home.txt << 'EOF'

---
Navigation: Home | About | Projects | Contact
---
EOF
```

Check status:

```
$ git status
```

Expected Output:

```
modified:   home.txt
```

Stage and commit:

```
$ git add home.txt
$ git commit -m "Add navigation section to homepage"
```

View log:

```
$ git log --oneline
```

Expected Output:

```
def5678 Add navigation section to homepage
abc1234 Add homepage with welcome message
```

Student Extension Challenge (10 minutes)

Add a footer to `home.txt`: `Built with Git`. Stage and commit it.

Solution

```
$ echo "" >> home.txt
$ echo "Built with Git" >> home.txt
$ git add home.txt
$ git commit -m "Add footer to homepage"
```

Verify:

```
$ git log --oneline
```

Expected: A third commit.

Session 2: Two New Pages — Branching for Features

The Goal Today

Add two new pages (About and Projects) using feature branches. Keep `main` clean and stable.

Theory: Why Branches Matter (15 minutes)

The Parallel Universe Analogy

> ***Analogy:*** *You're writing a book. The published version is* `main` *. You want to try a new chapter, but you're not sure if readers will like it. So you photocopy the book, write the new chapter in the copy, and show it to friends. If they like it, you paste it into the real book. If they hate it, you throw away the copy.*
>
> *The photocopy = a branch. The real book =* `main` *.*

Professional Context

In DevOps:
- `main` is **sacred** — always deployable
- Feature branches are **disposable** — experiment freely
- You merge only when code is tested and reviewed

Practical Build: About and Projects Pages (35 minutes)

Environment: Terminal (in `~/my-devops-portfolio`)

Ensure you're on `main`:

```
$ git branch
```

Expected Output:

```
* main
```

Feature 1: About Page

Create branch:

```
$ git branch feature-about-page
$ git switch feature-about-page
```

Check:

```
$ git branch
```

Expected Output:

```
* feature-about-page
  main
```

Build the About page:

```
$ cat > about.txt << 'EOF'
================================
  ABOUT ME
================================

Name: [Your Name]
Role: DevOps Student
Goal: To master Git, GitHub, and CI/CD

This portfolio tracks my learning journey.
EOF
```

Stage and commit:

```
$ git add about.txt
$ git commit -m "Add about page with bio"
```

Feature 2: Projects Page

Create a second branch (from `main`, not from the about branch):

First, go back to main:

```
$ git switch main
```

Create projects branch:

```
$ git branch feature-projects-page
$ git switch feature-projects-page
```

Build the Projects page:

```
$ cat > projects.txt << 'EOF'
================================
  MY PROJECTS
================================

1. DevOps Portfolio (this website!)
   - Tools: Git, GitHub, CI/CD
   - Status: In progress

More projects coming soon...
EOF
```

Stage and commit:

```
$ git add projects.txt
$ git commit -m "Add projects page"
```

Verify main is still clean:

```
$ git switch main
$ ls
```

Expected Output:

```
home.txt
```

(Only `home.txt` — the new pages are on branches, not main!)

Merge About Page First

```
$ git merge feature-about-page
```

Expected Output:

```
Fast-forward
 about.txt | 10 ++++++++++
```

Merge Projects Page

```
$ git merge feature-projects-page
```

Expected Output:

```
Merge made by the 'ort' strategy.
 projects.txt | 11 ++++++++++
```

Note: This may be a merge commit (not fast-forward) because `main` now has the About page.

Verify all files:

```
$ ls
```

Expected Output:

```
about.txt  home.txt  projects.txt
```

View the merge commit in log:

```
$ git log --oneline --graph
```

Expected Output:

```
*   merge123 Merge branch 'feature-projects-page'
|\
| * proj456 Add projects page
* | about789 Add about page with bio
|/
* def5678 Add navigation section
```

Clean up branches:

```
$ git branch -d feature-about-page
$ git branch -d feature-projects-page
```

Student Extension Challenge (10 minutes)

Create a `skills.txt` file on a new branch `feature-skills` with 3 skills listed. Merge it into `main` and delete the branch.

Solution

```
$ git branch feature-skills
$ git switch feature-skills
$ cat > skills.txt << 'EOF'
Skills:
- Git version control
- GitHub collaboration
- CI/CD pipelines
EOF
$ git add skills.txt
$ git commit -m "Add skills page"
$ git switch main
$ git merge feature-skills
$ git branch -d feature-skills
```

Verify:

```
$ ls
$ git log --oneline
```

Session 3: Going Online — GitHub and Pull Requests

The Goal Today

Push the portfolio to GitHub. Add a Contact page through a proper Pull Request workflow.

Theory: Sharing with the World (15 minutes)

The Library Analogy

> ***Analogy:*** *Your local Git repo is like a diary on your desk. GitHub is like a library where you publish your diary so others can read it. You push (upload) your finished chapters. You pull (download) edits from your editor.*

Why PRs?

You wouldn't publish a book without an editor reading it first. A Pull Request is like handing your chapter to the editor and saying "Is this good enough for the book?"

In DevOps:
- Pushing directly to `main` = editing the published book while it's in stores
- Opening a PR = submitting to the editor first

Practical Build: Contact Page via PR (35 minutes)

Environment: Browser + Terminal

Part 1: Create GitHub Repo

1. Go to `https://github.com`
2. Click **+** → **New repository**
3. Name: `my-devops-portfolio`
4. Public, no README
5. Click **Create**

Part 2: Push Local Code

Environment: Terminal

```
$ git remote add origin git@github.com:YOURUSERNAME/my-devops-portfolio.git
$ git push -u origin main
```

Expected Output:

```
 * [new branch]      main -> main
```

Refresh GitHub — your files are there!

Part 3: Contact Page via PR

Create branch:

```
$ git branch feature-contact-page
$ git switch feature-contact-page
```

Build the page:

```
$ cat > contact.txt << 'EOF'
================================
  CONTACT
================================

Email: your.email@example.com
GitHub: github.com/YOURUSERNAME

Let's connect!
EOF
```

Stage and commit:

```
$ git add contact.txt
$ git commit -m "Add contact page"
```

Push:

```
$ git push -u origin feature-contact-page
```

On GitHub:
1. Click **"Compare & pull request"**
2. Title: `Add contact page`
3. Description:
`- Added contact information - Follows existing page style`
4. **Create pull request**
5. **Files changed** → review
6. **Review changes** → **Approve** → **Submit**
7. **Merge pull request** → **Confirm**
8. **Delete branch**

Part 4: Sync Local Main

Environment: Terminal

```
$ git switch main
$ git pull
```

Expected Output:

```
From github.com:.../my-devops-portfolio
 ...
 contact.txt | 10 ++++++++++
```

Verify:

```
$ ls
```

Expected Output:

```
about.txt  contact.txt  home.txt  projects.txt  skills.txt
```

Clean up:

```
$ git branch -d feature-contact-page
```

Student Extension Challenge (10 minutes)

Update `projects.txt` to include a "Future Goals" section. Do it through a PR workflow: branch, edit, commit, push, PR, merge, pull.

Solution

```
$ git branch update-projects
$ git switch update-projects
```

Edit `projects.txt` to add:

```
Future Goals:
- Complete CI/CD certification
- Contribute to open source
- Build a cloud infrastructure project
```

```
$ git add projects.txt
$ git commit -m "Add future goals to projects page"
$ git push -u origin update-projects
```

On GitHub: Create PR, review, merge, delete branch. **Back in terminal:**

```
$ git switch main
$ git pull
$ git branch -d update-projects
```

Session 4: Fixing Mistakes and Handling The Unexpected

The Goal Today

Introduce a "bug" (wrong content), find it in history, revert it safely. Also handle a merge conflict when two branches change the same file.

Theory: When Things Go Wrong (15 minutes)

The Time Machine

`git log --oneline` is your time machine. It shows every snapshot. Find the bad one, then use `git revert` to undo it.

> ***Key rule:*** *We never erase history. We add to it. Revert creates a NEW commit that undoes an old one. This keeps the audit trail.*

Conflicts Are Normal

When two branches touch the same line, Git asks a human to choose. It's not scary — it's just Git saying "I need help deciding."

Practical Build: Bug Fix + Conflict Resolution (35 minutes)

Part 1: The Bug

Accidentally overwrite the homepage:

Environment: Terminal (on `main`)

```
$ echo "Oops, this is bad content." > home.txt
$ git add home.txt
$ git commit -m "Redesign homepage (broken)"
```

Check log:

```
$ git log --oneline
```

Find the bad commit ID (top one).

Revert it:

```
$ git revert HEAD
```

(HEAD means "the most recent commit on current branch")

An editor opens. Save and exit.

Expected Output:

```
[main revert123] Revert "Redesign homepage (broken)"
```

Verify the file:

```
$ cat home.txt
```

The original content is restored!

Push to GitHub:

```
$ git push
```

Part 2: The Merge Conflict

Create two branches that both edit `about.txt`:

```
$ git branch update-about-role
$ git switch update-about-role
```

Change the role line:

```
$ cat > about.txt << 'EOF'
================================
  ABOUT ME
================================

Name: [Your Name]
Role: DevOps Engineer (updated role)
Goal: To master Git, GitHub, and CI/CD

This portfolio tracks my learning journey.
EOF
$ git add about.txt
$ git commit -m "Update role on about page"
```

Now create another branch from `main`:

```
$ git switch main
$ git branch update-about-goal
$ git switch update-about-goal
```

Change the goal line:

```
$ cat > about.txt << 'EOF'
================================
  ABOUT ME
================================

Name: [Your Name]
Role: DevOps Student
Goal: To become a Senior DevOps Engineer

This portfolio tracks my learning journey.
EOF
$ git add about.txt
$ git commit -m "Update goal on about page"
```

Merge the first branch:

```
$ git switch main
$ git merge update-about-role
```

This works fine. Now merge the second:

```
$ git merge update-about-goal
```

Expected Output:

```
CONFLICT (content): Merge conflict in about.txt
Automatic merge failed
```

View the file:

```
$ cat about.txt
```

Expected Output:

```
<<<<<<< HEAD
Role: DevOps Engineer (updated role)
=======
Goal: To become a Senior DevOps Engineer
>>>>>>> update-about-goal
```

(Note: the actual conflict may include more context depending on Git version)

Resolve by combining both changes:

```
$ cat > about.txt << 'EOF'
================================
  ABOUT ME
================================

Name: [Your Name]
Role: DevOps Engineer (updated role)
Goal: To become a Senior DevOps Engineer

This portfolio tracks my learning journey.
EOF
```

Mark as resolved:

```
$ git add about.txt
$ git commit -m "Merge about updates and resolve conflict"
```

Clean up:

```
$ git branch -d update-about-role
$ git branch -d update-about-goal
$ git push
```

Student Extension Challenge (10 minutes)

Create a conflict on `contact.txt` between `main` and a new branch. Resolve it by keeping a merged version with both email and GitHub info enhanced.

Solution

```
$ git branch contact-update-1
$ git switch contact-update-1
$ echo "Email: my.email@example.com (preferred)" > contact.txt
$ git add contact.txt
$ git commit -m "Update email format"

$ git switch main
$ git branch contact-update-2
$ git switch contact-update-2
$ echo "GitHub: github.com/YOURUSERNAME (check out my repos!)" > contact.txt
$ git add contact.txt
$ git commit -m "Enhance GitHub link"

$ git switch main
$ git merge contact-update-1
$ git merge contact-update-2
# (conflict)

# Resolve
$ cat > contact.txt << 'EOF'
================================
  CONTACT
================================

Email: my.email@example.com (preferred)
GitHub: github.com/YOURUSERNAME (check out my repos!)

Let's connect!
EOF

$ git add contact.txt
$ git commit -m "Merge contact updates"
$ git branch -d contact-update-1
$ git branch -d contact-update-2
```

Session 5: Robots Do the Work — CI/CD with GitHub Actions

The Goal Today

Add a GitHub Actions workflow that automatically checks our portfolio whenever someone pushes code. Lock down `main` with branch protection.

Theory: Automation is DevOps (15 minutes)

The Robot Chef

> ***Analogy:*** *Imagine a restaurant where every time a chef adds a new dish to the menu, a robot automatically:*
> *1. Tastes the dish*
> *2. Checks the temperature*
> *3. Photographs it for social media*
> *4. Either approves it for the menu or throws it away*
>
> *GitHub Actions is that robot. Every time you push code, it runs tests automatically.*

Why This Matters

Manual Process	Automated Process
Developer pushes code	Developer pushes code
Team lead emails: "Did you test?"	Robot tests immediately
Bug found in production	Bug caught before merge
Blame and stress	Confidence and speed

Practical Build: CI Workflow (35 minutes)

Part 1: The Workflow File

Environment: Terminal (on `main`)

```
$ cd ~/my-devops-portfolio
$ git switch main
$ mkdir -p .github/workflows
```

Create the workflow:

```
$ cat > .github/workflows/portfolio-ci.yml << 'EOF'
name: Portfolio CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  check-portfolio:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Check required pages exist
        run: |
          for file in home.txt about.txt projects.txt contact.txt; do
            if [ -f "$file" ]; then
              echo "□ $file found"
            else
              echo "□ $file MISSING"
              exit 1
            fi
          done

      - name: Check for navigation keywords
        run: |
          if grep -q "Navigation" home.txt; then
            echo "□ Navigation section found"
          else
            echo "□ Navigation section missing"
            exit 1
          fi

      - name: Report success
        run: echo "Portfolio validation complete!"
EOF
```

Stage, commit, push:

```
$ git add .github/workflows/portfolio-ci.yml
$ git commit -m "Add CI workflow for portfolio validation"
$ git push
```

Environment: Browser
1. Go to the **Actions** tab on GitHub
2. Watch the workflow run
3. It should be green

Part 2: Test Failure

Create a branch that breaks the build:

```
$ git branch break-nav
$ git switch break-nav
$ cat > home.txt << 'EOF'
================================
  MY DEVOPS PORTFOLIO
================================

Navigation removed for testing.
EOF
$ git add home.txt
$ git commit -m "Remove navigation (this should fail CI)"
$ git push -u origin break-nav
```

On GitHub:
- Open the PR
- The workflow runs automatically
- It FAILS because "Navigation" keyword is missing
- Show students the failed step output
- Close PR without merging

Part 3: Branch Protection

1. GitHub repo → **Settings** → **Branches**
2. Add rule: `main`
3. Check:
   - **Require pull request before merging**
   - **Require status checks to pass**
   - Select `Portfolio CI`
4. **Create**

Verify by trying direct push:

```
$ git switch main
$ echo "Direct push test" >> home.txt
$ git add home.txt
$ git commit -m "Test direct push"
$ git push
```

Expected: Error — branch is protected.

Undo:

```
$ git reset --soft HEAD~1
$ git restore --staged home.txt
$ git checkout -- home.txt
```

Student Extension Challenge (10 minutes)

Add a check to the CI that verifies `skills.txt` exists. Push it and watch it run.

Solution

Edit `.github/workflows/portfolio-ci.yml` and add inside the `run:` block of "Check required pages exist":

```
      - name: Check required pages exist
        run: |
          for file in home.txt about.txt projects.txt contact.txt skills.txt; do
            if [ -f "$file" ]; then
              echo "$file found"
            else
              echo "$file MISSING"
              exit 1
            fi
          done
```

Or via terminal (be careful with indentation):

```
$ git switch main
# edit file manually in a text editor
$ git add .github/workflows/portfolio-ci.yml
$ git commit -m "Extend CI to check skills page"
$ git push
```

Watch it in the Actions tab.

Final Deliverable Checklist

By the end of Session 5, every student should have:

- [ ] A GitHub repo at `github.com/YOURUSERNAME/my-devops-portfolio`
- [ ] At least 5 text files (pages)
- [ ] A commit history with meaningful messages
- [ ] At least 2 merged pull requests
- [ ] A passing GitHub Actions workflow
- [ ] Branch protection enabled on `main`

This repo is their proof of DevOps skills for a job interview!

Appendix: Quick Reference

Command	What It Does
`git init`	Start version control
`git status`	See what's changed
`git add <file>`	Stage changes
`git commit -m "msg"`	Save a snapshot
`git log --oneline`	View history
`git branch <name>`	Create branch
`git switch <name>`	Change branch
`git merge <name>`	Combine branches
`git push -u origin <branch>`	Upload branch
`git pull`	Download latest
`git revert <commit>`	Undo safely
`git stash` / `git stash pop`	Temporary save

End of Approach 2: Project-Driven Build-Along